



QpiAI™ - Joint AI Training and Certification Program

Feature Engineering, Hyperparameter Tuning

Sachin Kumar
Arpit Jain, PhD

Recap:

Basic Probability &
Statistics



Regression and
Classification

Linear & Logistic
Regression

K-Nearest Neighbor
Algorithm

KNN

SVM (SVC)
Support Vector
Machines, Classification
Performance Metrics

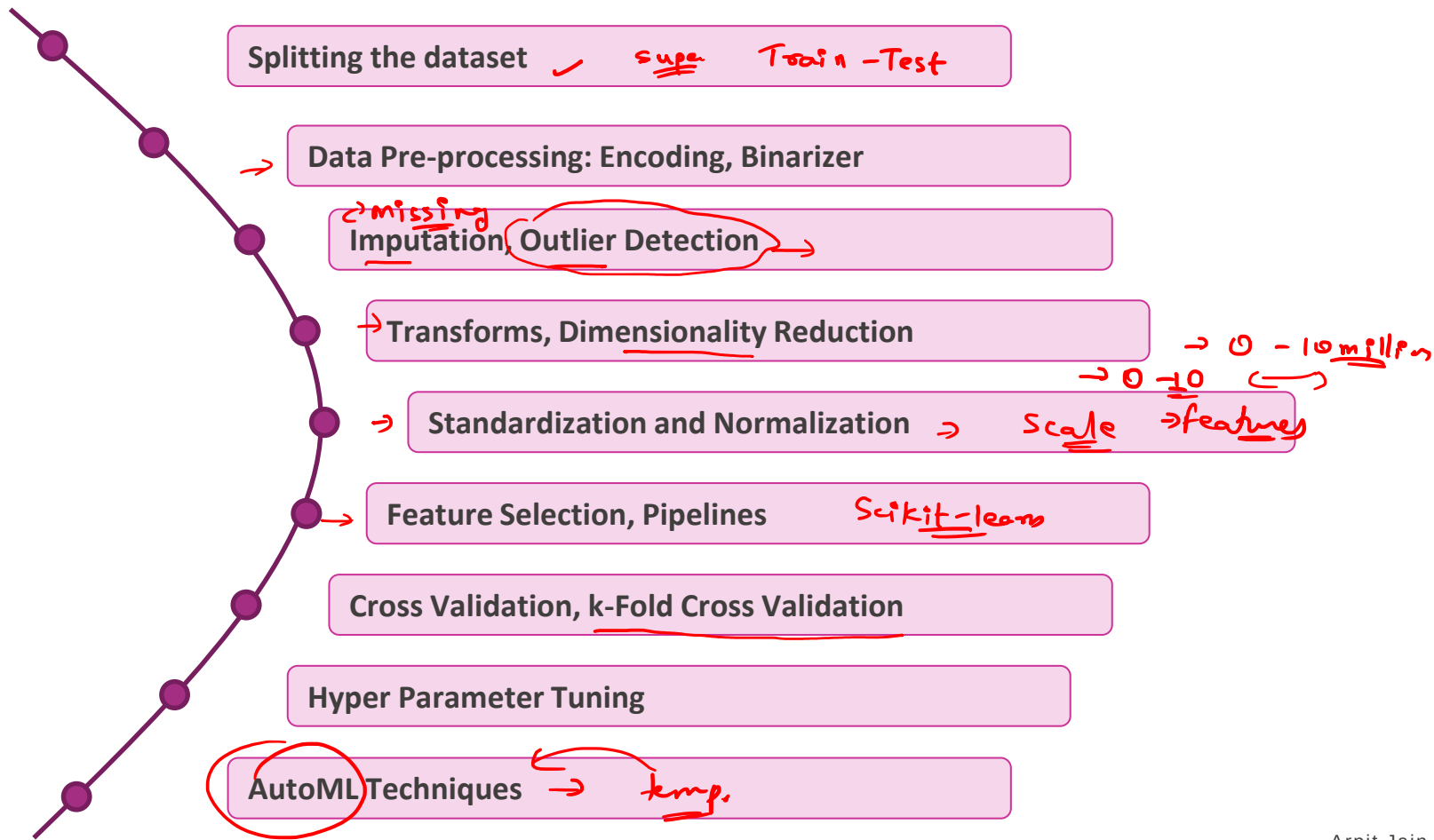
Tree
Decision Tree,
Random Forest &
Ensemble Methods

PCA
Principal Component
Analysis and k- Means

Time Series Analysis

Unsupervised

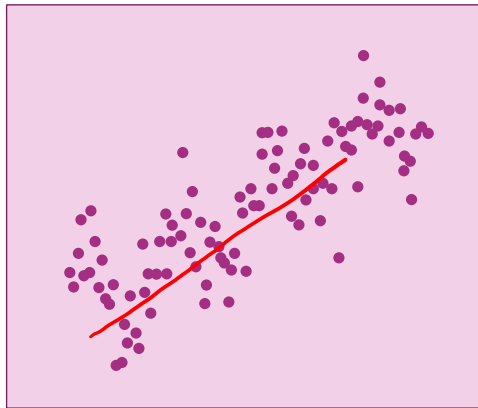
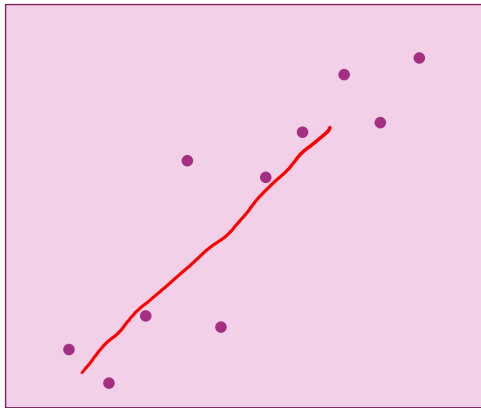
Overview



More the data, better in general predictions get

Regression

$f(x) \uparrow$
depe



$\rightarrow x$
indep

$\rightarrow$ overfitting

Train Test Split (Random and Stratified Sampling)

↳ Algo ↳ verify

↳ Imbalanced →

Now we've understood how the models work. How do we know that it will work on properly any outside data other than the training dataset?

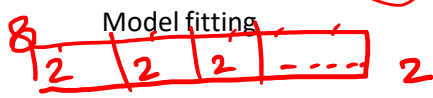
To know it better we don't train on model on the entire dataset, instead we take a split of it. On one the split we train our model (train set) and the other split is emulated as the outside world data on which we evaluate the model (test set).

Usually on a large dataset we usually split such a way that 80% of datapoints are in train set and the rest 20% is test set.

Some classification problems do not have a balanced number of examples for each class label. As such, it is desirable to split the dataset into train and test sets in a way that preserves the same proportions of examples in each class as observed in the original dataset. This is when we use Stratified Sampling.

equal distribution

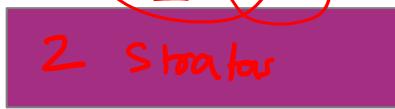
Training set



Classification
Binary
→ Yes → 1 → 20%
→ No → 0 → 80%

30%
25%

Test set
46 → 0
4 → 1



Evaluation

Identifying Model Performance Sweet Spot

Regressi

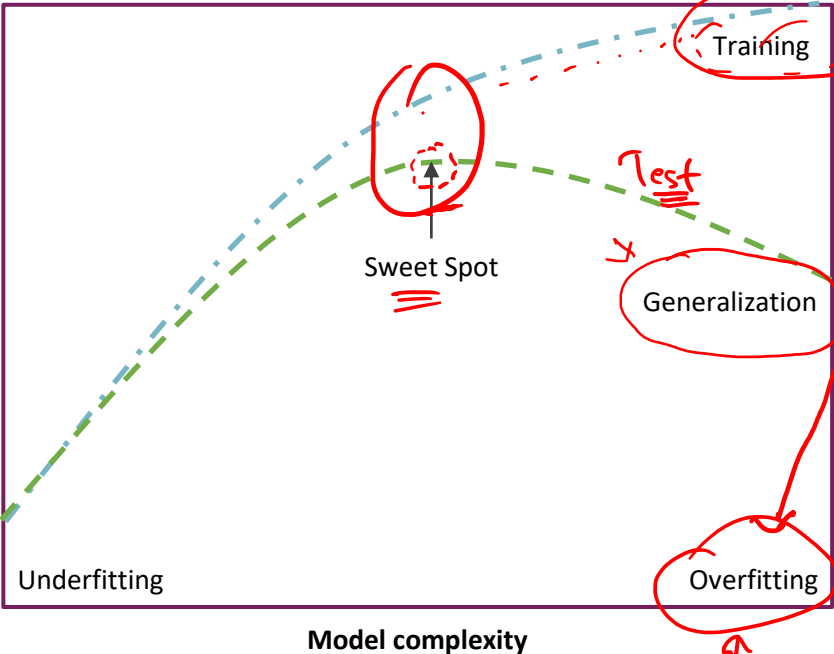
choose

→ Train ✓
→ Test ✓

Classif

Perfor
→ learn

Accuracy



Sweet Spot

Generalization

→ Test | Real-world

Overfitting

Types of Data

→ Categorical

ML → numerical

Nominal

A variable with categories that does not have a natural order or ranking. Can code nominal variables with numbers, but the order is arbitrary and any calculations, such as computing a mean, median, or standard deviation, is meaningless. Eg: blood group, zip code, ethnicity, eye color.

→ Ordinal

An ordinal scale is one where the order matters but not the difference between values. Eg:

- Socio economic status: low income, middle income, high income
- Education level: high school, undergraduate, graduate, doctoral

→ Interval

A variable following interval scale is one where there is order and the difference between two values is meaningful.

Eg: Temperature in Celsius, pH, Credit Score.

→ Ratio

A ratio variable, has all the properties of an interval variable, and also has a clear definition of 0. When the variable equals 0, there is none of that variable. Eg: Weight, Temperature in Kelvin.

1
→ Brown, Black, ...

↓
0 + ve B + ve
0 ps B ps

1 (-) 2 (-) 3 (-) 4 (-)
Ranking

Ranking/order ✓

7.0 8.5 9.2
Ranking

0 → no object

Dataset Encoding

Categorical - numerical

- Splitting the dataset doesn't always ensure whether it is suitable for training with the model.
- Models are mathematical tools and expects only numerical values in the dataset.
- But datasets doesn't always have numerical values only. It has all kind of categorical, string etc. valued columns. So we need to perform various kinds of dataset encoding so that it is in readable format by the model.
- There are various kinds of Dataset encodings we can perform like label encoding, one-hot encoding, label binarizer, etc.
- Label Binarizer: Binarizes data (set feature values to 0 or 1) according to a threshold (default 0), values greater than the threshold map to 1, while values less than or equal to the threshold map to 0. With the default threshold of 0, only positive values map to 1.

Yes → 1
No → 0

multi-class

One Hot Encoding

→ get dummies

n → adds n columns

↳ Scikit-learn

It refers to splitting the column which contains numerical categorical data to many columns depending on the number of categories present in that column. Each column contains "0" or "1" corresponding to which column it has been placed.

Foundation	Type_is_BrkTil	Type_is_CBlock	Type_is_PConc	Type_is_Slab	Type_is_Stone	Type_is_Wood
PConc	0	0	1	0	0	0
CBlock	0	1	0	0	0	0
PConc	0	0	1	0	0	0
BrkTil	1	0	0	0	0	0
PConc	0	0	1	0	0	0
Wood	0	0	0	0	0	1
PConc	0	0	1	0	0	0
CBlock	0	1	0	0	0	0
BrkTil	1	0	0	0	0	0

Handwritten notes and annotations:

- 1st → 4 unique labels
- 2nd → 2nd
- 3rd → 3rd
- 4th → 4th
- 5th → 5th
- 6th → 6th
- nominal
- model
- is
- adding n few columns
- dev Train / test
- deployed
- 1000
- 1001
- slab / stone
- abc

Label Encoding → Change of Dimension is not there

Label Encoder encodes labels within a categorical column with a value between 0 and n_classes - 1 where n_classes is the number of distinct labels.

Label encoding can work on Numeric data too, but it is preferable to use with Categorical data.

	Foundation	Foundation_label_encoded
2	PConc	2
1	CBlock	1
	PConc	2
0	BrkTil	0
	PConc	2
3.5	Wood	5 ✓
	PConc	2
	CBlock	1
	BrkTil	0

3 → Slab
4 → Stone

1000
↳
1500
↳
2000
2001 → Stone / Slab

4 | 3 → Stone / Slab

→ model

←

○ ○ ○ ○ ○

Imputation

→ missing → dataset

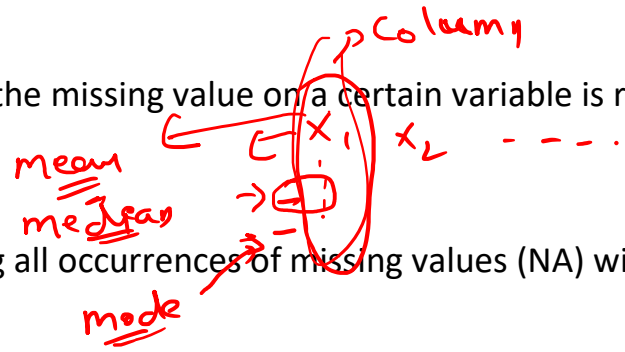
Replace → compute

→ KNN Imputation

This imputer utilizes the k-Nearest Neighbors method to replace the missing values in the datasets with the mean value from the parameter 'n_neighbors' nearest neighbours found in the training set. By default, it uses a Euclidean distance metric to impute the missing values.

Mean Imputation →

Mean imputation is a method in which the missing value on a certain variable is replaced by the **mean** of the available cases.

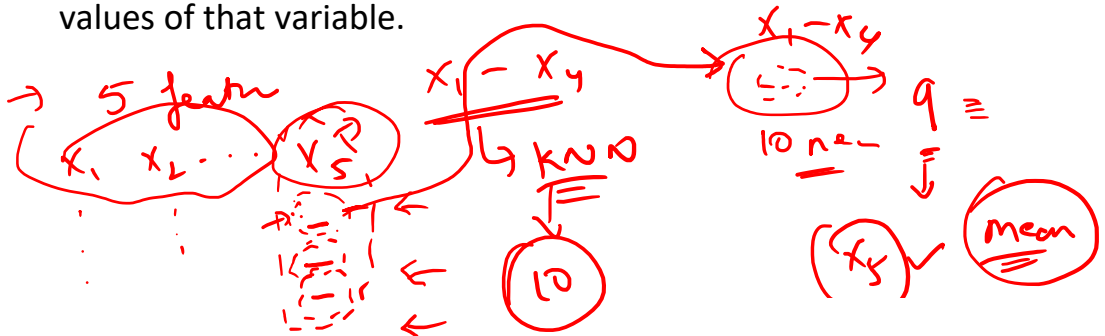


- Median Imputation

Median imputation consists of replacing all occurrences of missing values (NA) within a variable by its **median**.

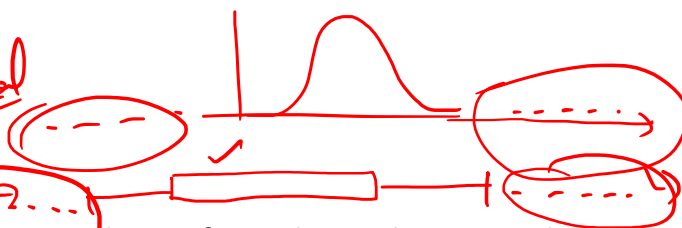
- Mode Imputation

Mode imputation replaces missing values of a categorical variable by the **mode or most common** of non-missing values of that variable.



Outlier Detection

Visual



dropped $\rightarrow$ input
outliers

Outlier Detection Through Interquartile

Helps us eliminate the outliers which are 1.5 times lower from the 25th percentile or 1.5 times above the 75th percentile. Works only with continuous values.

0% 25% 50% 75% 100%

threshold

outliers

Outlier Detection Through Z score

z-score

Helps eliminate the outliers which are 'n' standard deviations away from the mean. A column with continuous values should be given as inputs. Before applying this feature all columns should have values filled and there should be no infinite value.

Outlier Detection Through Isolation Forest

Random forest $\rightarrow$

outliers

$\hookrightarrow$ first few splits

Helps eliminate multiple anomalies and outliers present in the dataset. It uses an algorithm similar to Random Forest. Multiple columns with continuous values can be given as inputs. Before applying this feature all columns should have values filled and there should be no infinite value. There are three types of Isolation Forest algorithms we have applied Isolation Forest Univariate, Isolation Forest Bi-variate, Isolation Forest Tri-variate.

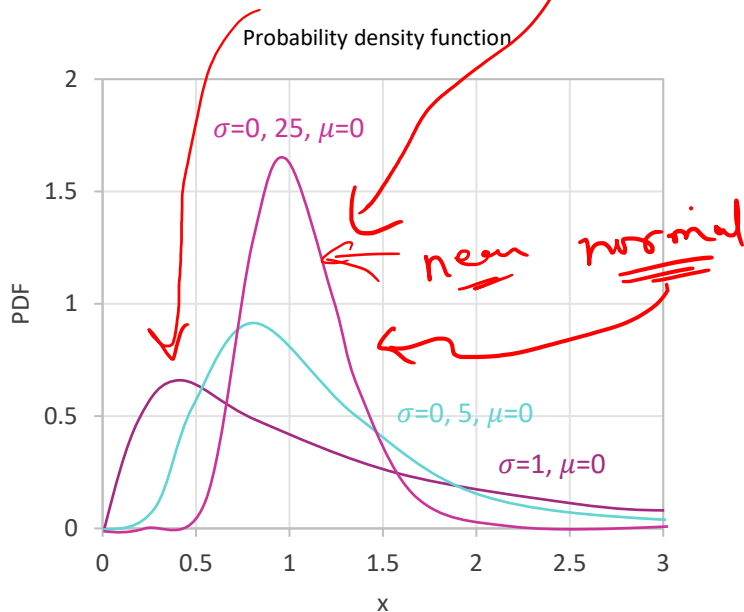
$\rightarrow$ missing

$\rightarrow$ infinite

Log-Normal Transform

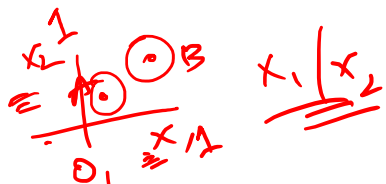
assumpt line by
→ Features - Normal

- When a numerical feature does not follow the normal distribution, we can log transform this data to make it as normally distributed as possible so that the statistical analysis results from this data become more valid.
- In other words, the log transformation reduces or removes the skewness of our original data, if it approximately follows a log-normal distribution.



$$\ln(X) \sim N(\mu, \sigma^2)$$

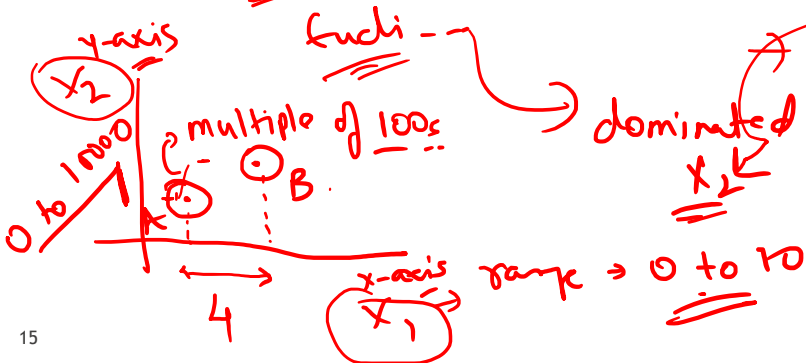
Standardization is another scaling technique where the values are centered around the mean with a unit standard deviation by computing Z-score. This means that the mean of the attribute becomes zero and the resultant distribution has a unit standard deviation.



Min-Max Scaling

ensures

Normalization is a scaling technique in which values are shifted and rescaled so that they end up ranging between 0 and 1. It is also known as Min-Max scaling.



$$X_N = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

$$(200)^2 > (4)^2$$

mean (50)

x1

x2

0-10

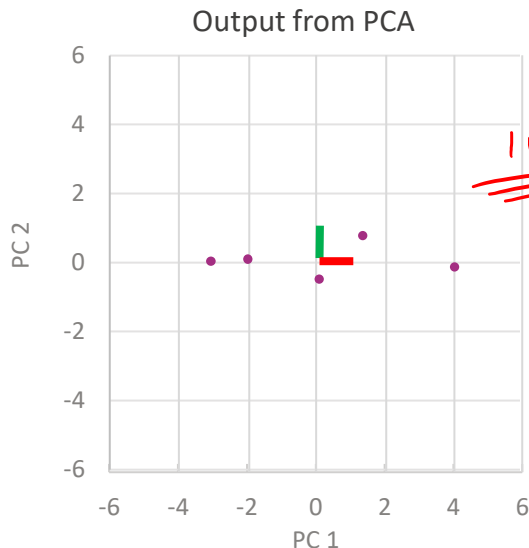
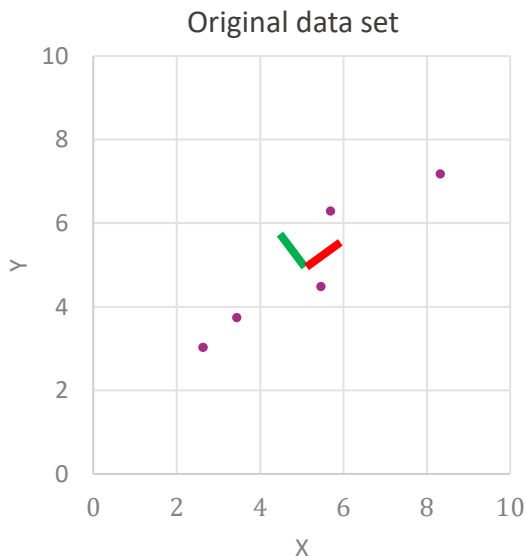
0-million

k ~ n

PCA: Dimensionality Reduction Intuition

curse of Dimensionality

Principal component analysis (PCA) is the process of computing the principal components and using them to perform a change of basis on the data, using only the first few principal components since they capture the maximum variance within the dataset.



1 million features

Columns

10000

→ 90%.

~~90000~~

95%.

PC

1

n

PC

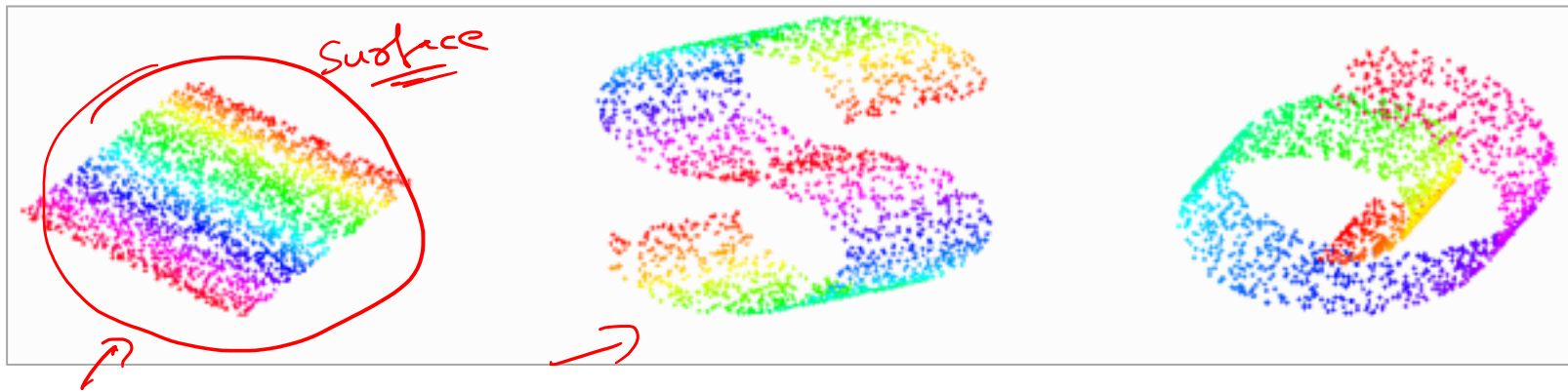
fea

→

Manifold Learning

→ surface

Similarly, Manifold Learning can find lower dimension representation of data. In the following cases, reducing the dimension of data from 3 dimensions to 2 dimensions, since the data lies in 3D, but can be unfolded into 2D plane.



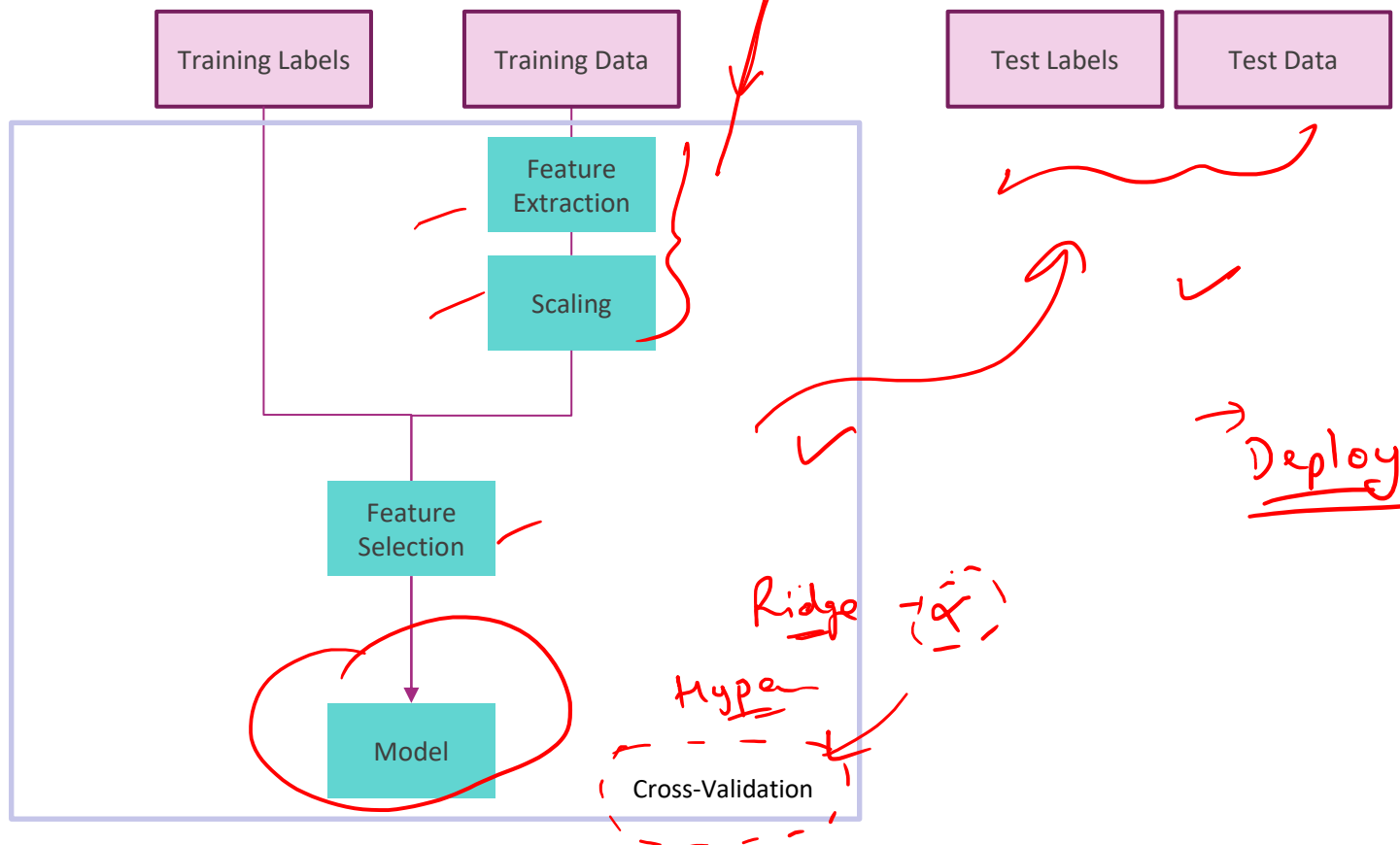
Model Pipeline

Hyper

Train

Test

FE
↓
Caterg-
Scale
missi
outl
↓
Dataset

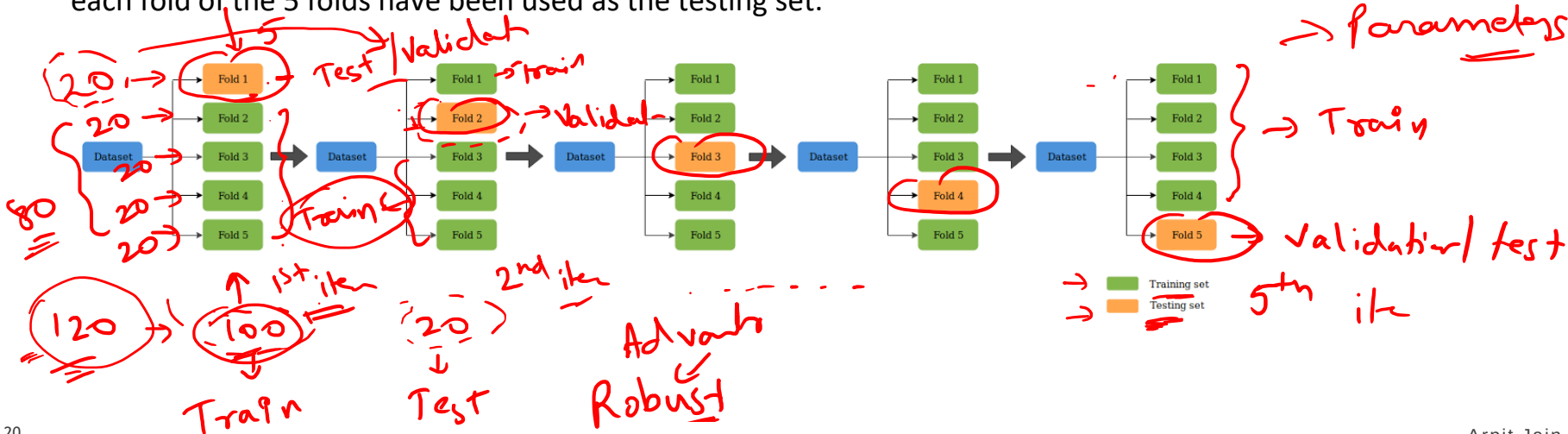


Deploy

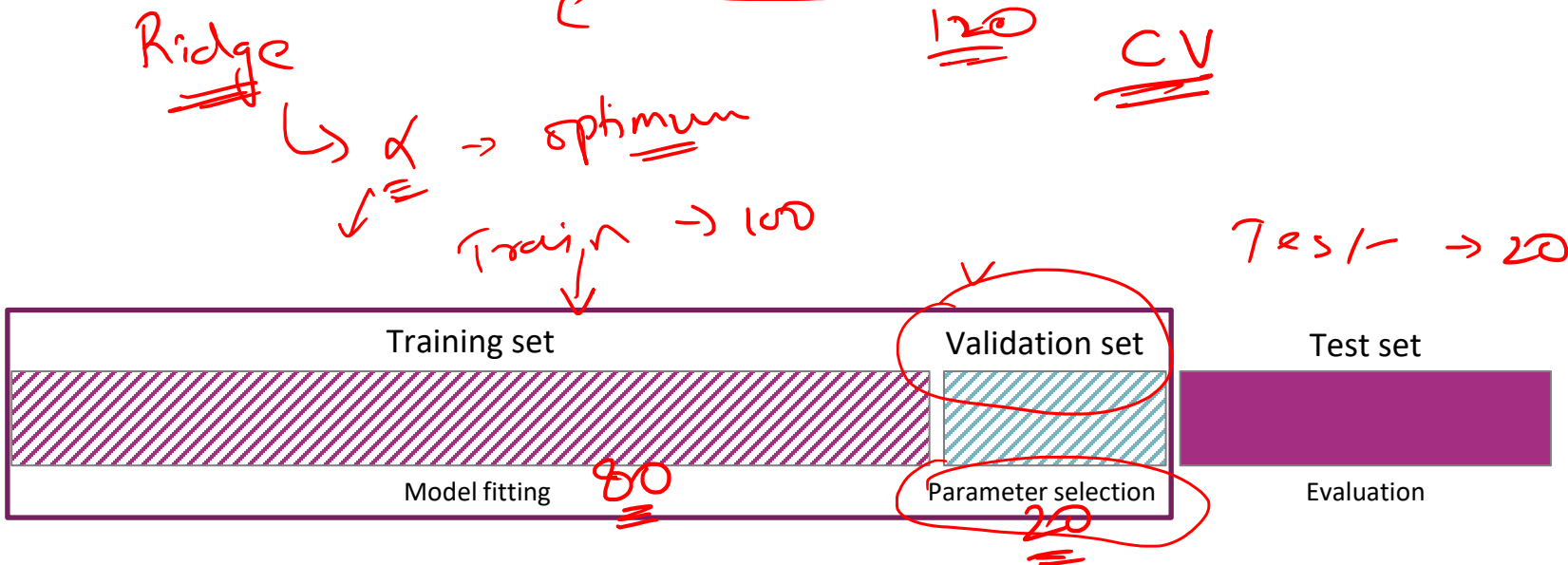
Cross-validation is any of various similar model validation techniques for assessing how the results of a statistical analysis will generalize to an independent data set. It is mainly used in settings where the goal is prediction, and one wants to estimate how accurately a predictive model will perform in practice.

K-Fold Cross Validation

K-Fold CV is where a given data set is split into a K number of sections/folds where each fold is used as a testing set at some point. Let's take the scenario of 5-Fold cross validation ($K=5$). Here, the data set is split into 5 folds. In the first iteration, the first fold is used to test the model and the rest are used to train the model. In the second iteration, 2nd fold is used as the testing set while the rest serve as the training set. This process is repeated until each fold of the 5 folds have been used as the testing set.

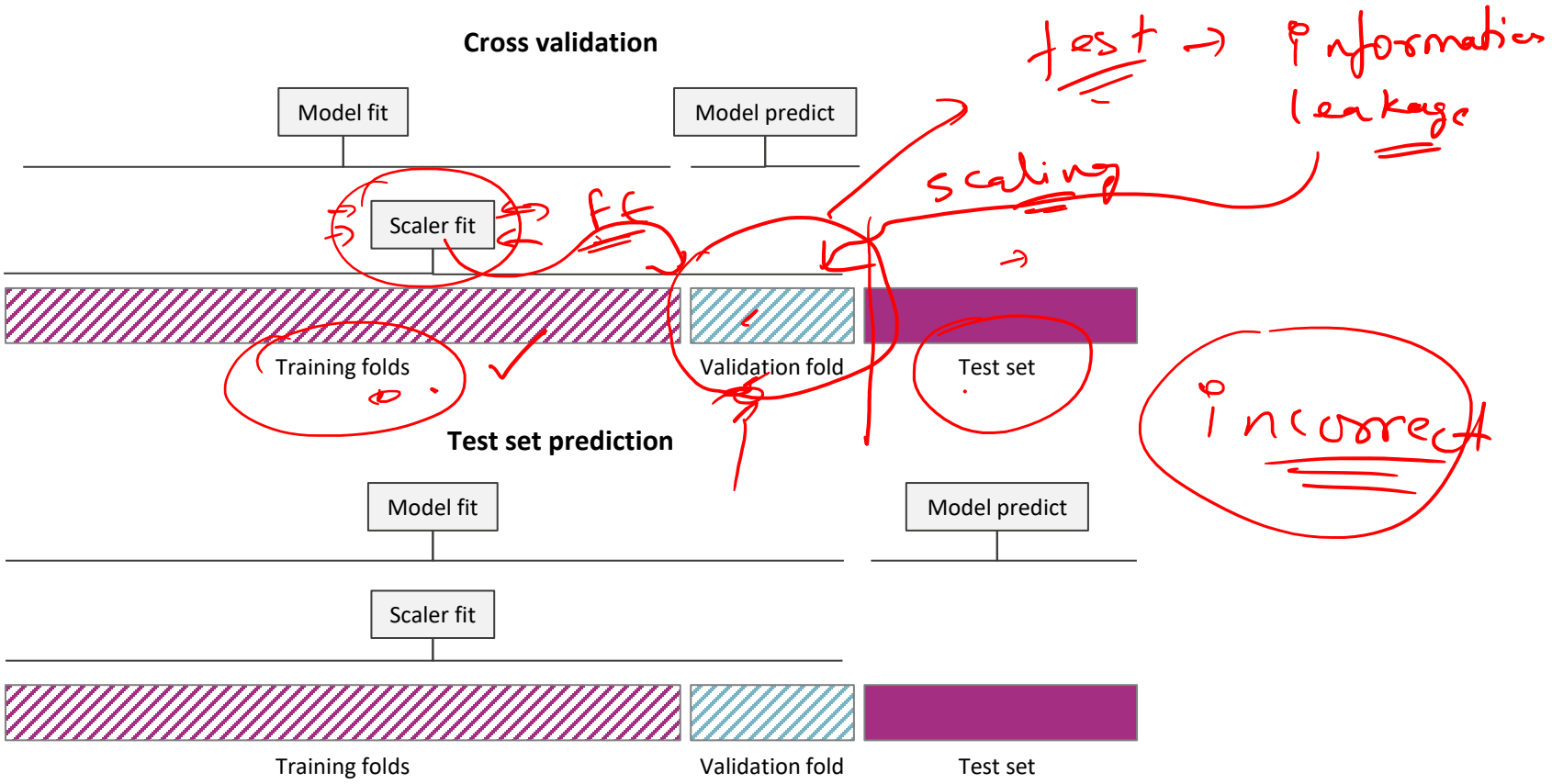


Dividing Dataset for Hyper Parameter Tuning

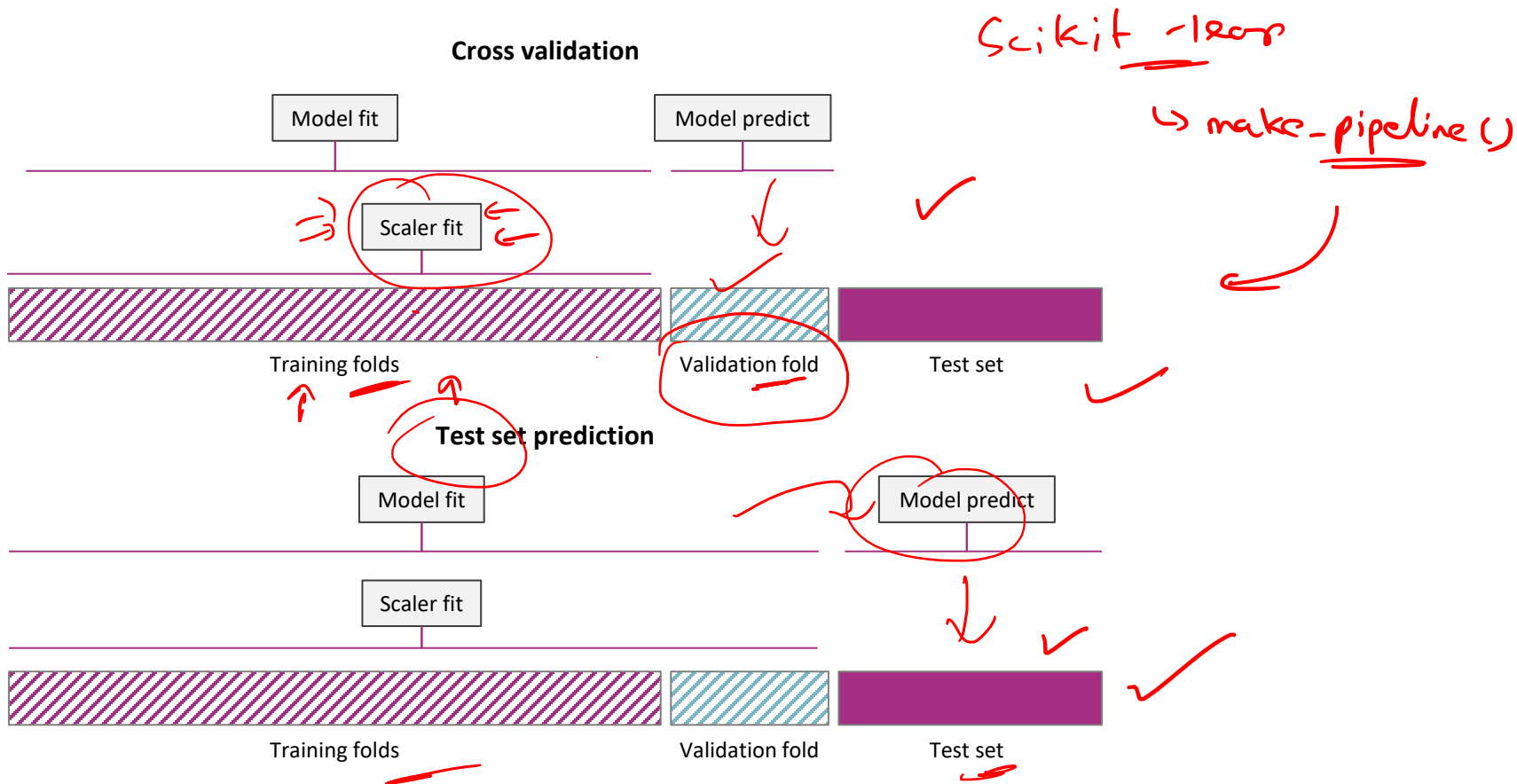


K -> fold
fold -> CV
5 x fold

Encouraging Pipeline in Cross Validation

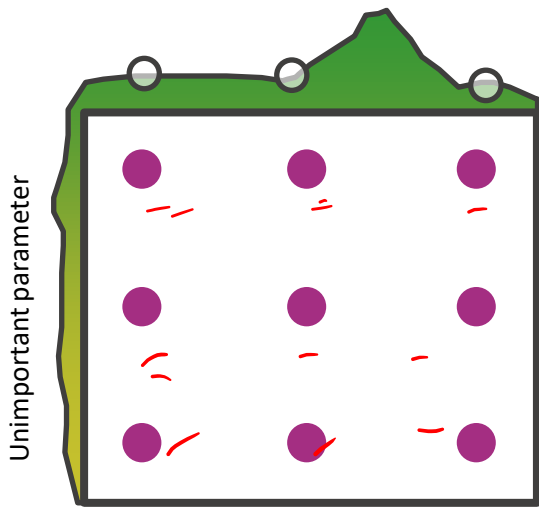


Correct Pipeline for Cross Validation



Grid Search v/s Random Search

Grid Layout



Unimportant parameter

Important parameter

parameter
grid

optimum

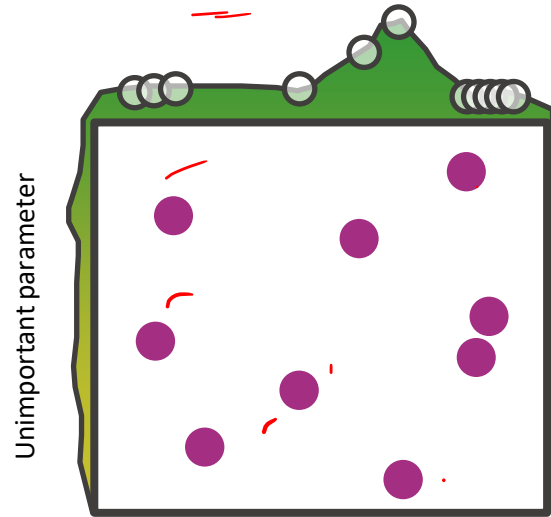
$$\alpha = [0, 1, 1, 0, 1, 0, 2]$$

Grid Search CV()

return

α

Random Layout



Unimportant parameter

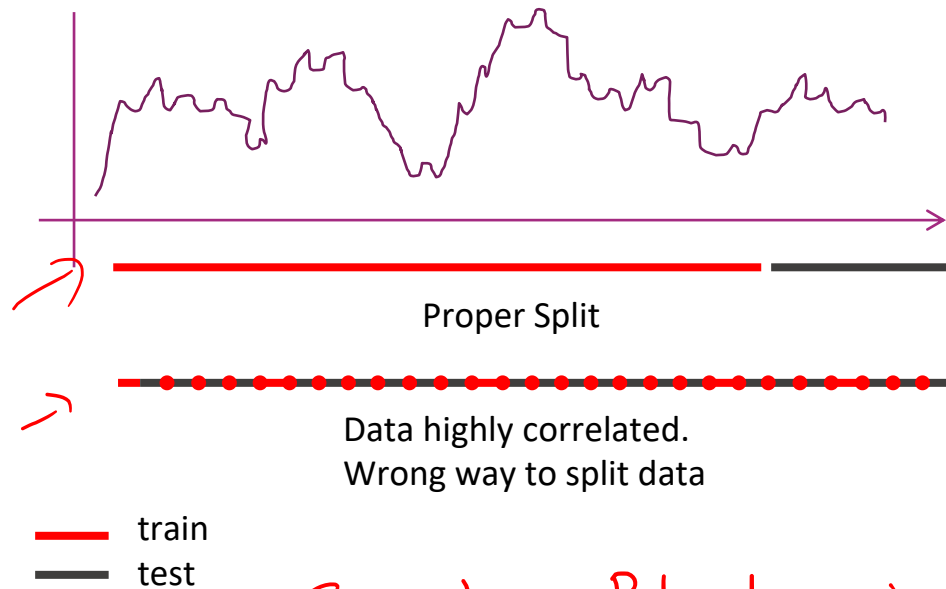
Important parameter

Perf opti (highest)

Ridge $\rightarrow$ α

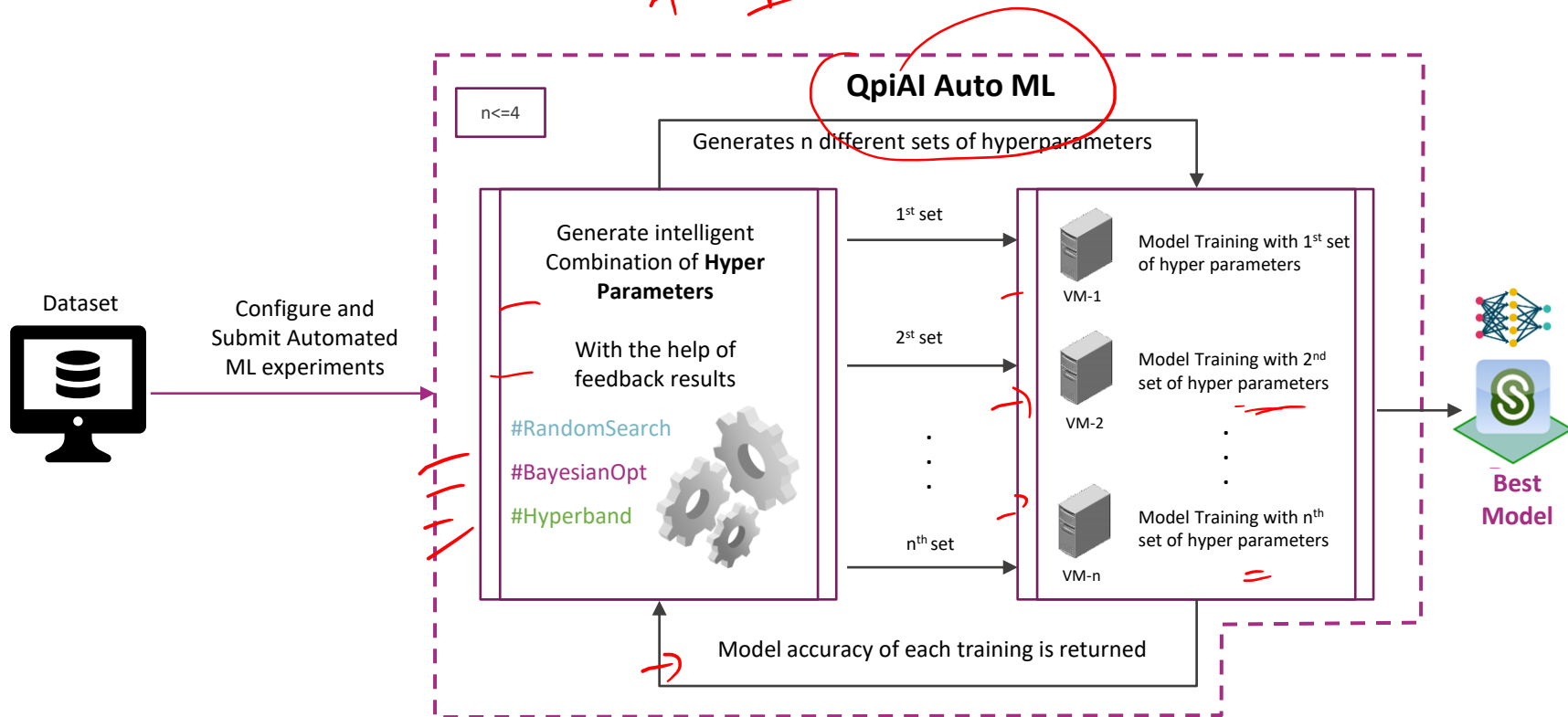
Evaluation and Cross-Validation with Time Series Data

Since time series data is highly correlated, one cannot sample test data at random but is forced to sample train-validate- test data block-wise.



Sample $\rightarrow$ Block-wise

level. --





$\sin \theta = \frac{OPP}{HYP}$
 $\cos \theta = \frac{ADJ}{HYP}$
 $\tan \theta = \frac{OPP}{ADJ}$
 $\csc \theta = \frac{HYP}{OPP}$
 $\sec \theta = \frac{HYP}{ADJ}$
 $\cot \theta = \frac{ADJ}{OPP}$

$1 = \sin^2 \theta + \cos^2 \theta$
 $1 = \sec^2 \theta - \tan^2 \theta$
 $1 = \csc^2 \theta - \cot^2 \theta$

$\sin \frac{\theta}{2} = \pm \sqrt{\frac{1 - \cos \theta}{2}}$
 $\cos \frac{\theta}{2} = \pm \sqrt{\frac{1 + \cos \theta}{2}}$
 $\tan \frac{\theta}{2} = \pm \sqrt{\frac{1 - \cos \theta}{1 + \cos \theta}}$

$\sin 2\theta = \frac{2 \tan \theta}{1 + \tan^2 \theta}$
 $\cos 2\theta = \frac{1 - \tan^2 \theta}{1 + \tan^2 \theta}$
 $\tan 2\theta = \frac{2 \tan \theta}{1 - \tan^2 \theta}$

$\sin \theta = \pm \sqrt{1 - \cos^2 \theta}$
 $\cos \theta = \pm \sqrt{1 - \sin^2 \theta}$
 $\tan \theta = \pm \frac{\sin \theta}{\cos \theta}$

$F_x = F \cos \theta$
 $F_y = F \sin \theta$
 $F = \sqrt{F_x^2 + F_y^2}$
 $\theta = \tan^{-1} \left(\frac{F_y}{F_x} \right)$

$\oint \vec{E} \cdot d\vec{l} = -\frac{d\phi_B}{dt}$
 $\oint \vec{B} \cdot d\vec{l} = \mu_0 I + \mu_0 \epsilon_0 \frac{d\phi_E}{dt}$
 $\phi_B = BA = B h_1 dx$
 $\frac{d\phi_B}{dt} = h_1 dx \frac{dB}{dt}$
 $\oint \vec{E} \cdot d\vec{l} = -h_1 dx \frac{dB}{dt}$
 $h_1 dE = -h_1 dx \frac{dB}{dt}$
 $\frac{dE}{dx} = -\frac{dB}{dt}$
 $\frac{\partial E}{\partial x} = -\frac{\partial B}{\partial t}$
 $\frac{\partial^2 E}{\partial x^2} = \mu_0 \epsilon_0 \frac{\partial^2 E}{\partial t^2}$
 $E = E_{max} \sin(\omega t - kx)$
 $\frac{\partial E}{\partial x} = -k E_{max} \cos(\omega t - kx)$

$I_E = I_B + I_C$ and $\frac{I_C}{I_E} = \alpha$
 $I_B = I_E - I_C$
 $I_B = I_E (1 - \alpha)$
 $\beta = \frac{I_C}{I_B} = \frac{\alpha}{1 - \alpha}$

$V_{CC} = 10V$
 $R_1 = 1k\Omega$
 $R_2 = 1.5k\Omega$
 $R_3 = 100\Omega$
 $R_4 = 100\Omega$
 $R_5 = 2k\Omega$
 $C_1 = 10\mu F$
 $C_2 = 10\mu F$
 $C_3 = 490\mu F$
 $Q_1 = 2N3904$
 $R_L = 22\Omega$

$M = \frac{f}{s - f} = \frac{s' - f}{f}$
 $\frac{1}{f} = \frac{1}{s} + \frac{1}{s'}$
 $m = \frac{i}{o} = \frac{s'}{s}$
 $f = \frac{R}{2}$

$\Delta x = d \sin \theta$
 $d \sin \theta = m\lambda$
 $\sin \theta = \frac{m\lambda}{d}$
 $\theta = \sin^{-1} \left(\frac{m\lambda}{d} \right)$

$P_o = V_o I_o \cos \theta_o$
 $\cos \theta_o = \frac{P_o}{V_o I_o}$
 $I_c = I_o \cos \theta_o$, $I_m = I_o \sin \theta_o$
 $R_c = \frac{V_o^2}{P_o}$, $X_m = \frac{V_o}{I_m}$
 $\rho = \frac{m}{V}$

$n \text{ HOOC(CH}_2)_6\text{COOH} + n \text{ H}_2\text{N(CH}_2)_6\text{NH}_2 \rightarrow \text{Nylon 6-6} + 2n \text{ H}_2\text{O}$

$\text{CH}_3\text{CH}_2\text{COOH} + \text{CH}_3\text{CH}_2\text{OH} \rightarrow \text{CH}_3\text{CH}_2\text{COOCH}_2\text{CH}_3 + \text{H}_2\text{O}$

$n(A \cup B) = n(A) + n(B) - n(A \cap B)$
 $n(A \cup B \cup C) = n(A) + n(B) + n(C) - n(A \cap B) - n(A \cap C) - n(B \cap C) + n(A \cap B \cap C)$

Thank you Q&A